

Plantilla de Informe de Trabajo Práctico

1. Carátula

- **Título del trabajo:** TRABAJO PRÁCTICO N° 1: Aplicaciones de TADs
- **Integrantes del equipo:**
 - Bautista Kadur El Ainie
 - Fausto Siviero
- **Fecha de entrega:** 28/09/2026
- **Link del repositorio:**
<https://github.com/FaberCasio/AyED2026c2-Briggiler-Kadur-El-Ainie-Siviero.git>

2. Introducción

El trabajo práctico tiene como **objetivo** aplicar, analizar y evaluar experimentalmente diversas estructuras de datos y algoritmos fundamentales para la resolución eficiente de problemas computacionales.

Se buscan resolver tres **problemas** concretos:

- **Gestión dinámica de datos en memoria (Proyecto 1):** Manipular secuencias de elementos sin limitaciones de tamaño fijo de los arreglos estáticos, permitiendo inserciones, eliminaciones y recorridos eficientes en ambas direcciones mediante la Lista Doblemente Enlazada (LDE).
- **Planificación equitativa de recursos en un sistema operativo (Proyecto 2):** Asignar el uso de la CPU entre múltiples procesos concurrentes de forma justa mediante el algoritmo Round Robin, utilizando una Cola Circular para gestionar la secuencia de turnos y evitar la inanición de los procesos.
- **Organización eficiente de la información (Proyecto 3):** Ordenar colecciones de datos numéricos desordenados para optimizar su posterior acceso y procesamiento, evaluando qué algoritmo de ordenamiento (Burbuja, Quicksort o Radix Sort) resulta más conveniente según el tamaño de la entrada.

Los **temas teóricos** que se aplican son:

- **Estructuras de datos lineales:** Listas doblemente enlazadas y colas circulares FIFO.
- **Análisis de algoritmos:** Análisis *a priori* (notación O-grande) y análisis *a posteriori* (medición empírica de tiempos de ejecución).
- **Estrategias de ordenamiento:** Comparación e intercambios ($O(n^2)$), divide y vencerás ($O(n \log n)$) y procesamiento por dígitos no comparativo ($O(d \cdot n)$).
- **Planificación de procesos:** Administración de tiempo de CPU mediante asignación de *quantum* en esquemas Round Robin.

Plantilla de Informe de Trabajo Práctico

3. Desarrollo de los problemas

3.1. Problema N° 1: Lista Doblemente Enlazada (LDE)

A. Análisis y planteo

- **Descripción abstracta:** Gestionar una secuencia lineal de elementos donde cada nodo posee un puntero al siguiente y otro al anterior, permitiendo la manipulación dinámica en memoria sin requerir tamaño fijo.
- **Operaciones críticas:** Inserción al inicio/final, eliminación de un nodo específico, búsqueda y recorrido en ambas direcciones.
- **Estructura elegida:** Lista Doblemente Enlazada (LDE).
- **Supuestos:** Satisface la necesidad de modificar el tamaño de la estructura en tiempo de ejecución con costo $O(1)$ para inserciones y eliminaciones cuando ya se tiene la referencia al nodo, superando las limitaciones de los arreglos contiguos.

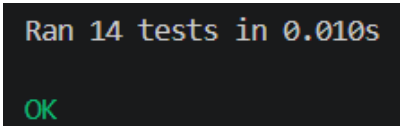
B. Implementación / Resolución

Se definió la clase `Nodo` (con `dato`, `siguiente` y `anterior`) y la clase `ListaDoblementeEnlazada` con sus métodos principales como por ejemplo: `agregar_inicio`, `agregar_final`, `insertar`, `extraer`, `copiar`, `invertir`... y recorridos.

C. Resultados y pruebas

Al correr el **código** de la Lista Doblemente Enlazada, se verifica que la prueba para los métodos funciona correctamente con la siguiente salida: [3 <-> 2 <-> 1 <-> 4 <-> 5 <-> 6]

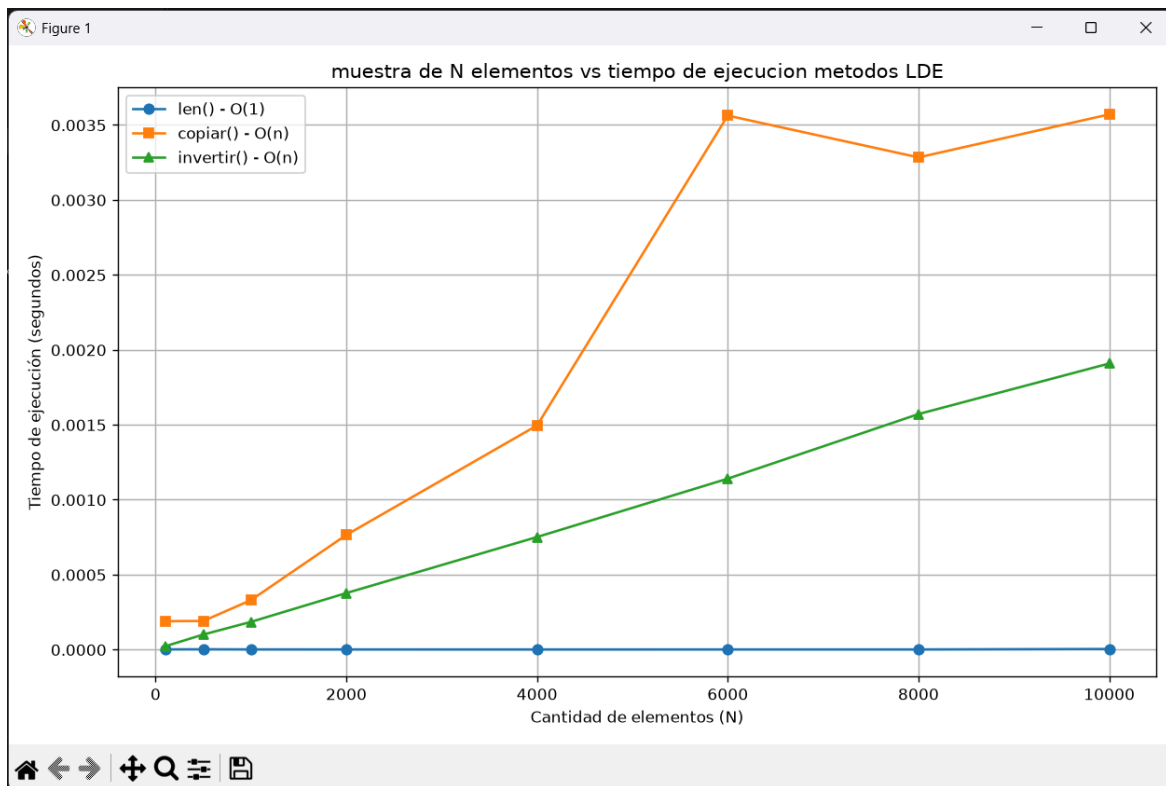
También se comprobó que corran los **test** de la LDE:



```
Ran 14 tests in 0.010s
OK
```

Gráfica de los tiempos de ejecución de las operaciones:

Plantilla de Informe de Trabajo Práctico



3.2. Problema N° 2: Planificación de Procesos (Round Robin)

A. Análisis y planteo

- **Descripción abstracta:** Simular la asignación justa de tiempo de CPU entre un conjunto de procesos concurrentes usando un intervalo fijo de tiempo denominado *quantum*.
- **Operaciones críticas:** Encolar procesos nuevos, desencolar el proceso en turno, descontar su tiempo de ejecución y reencolarlo si aún requiere CPU.
- **Estructura elegida:** Cola Circular FIFO.
- **Supuestos:** La estructura circular evita el desperdicio de memoria y permite un ciclo continuo de turnos, garantizando que ningún proceso sufra de inanición (*starvation*).

B. Implementación / Resolución

Se implementó una Cola Circular mediante punteros o arreglos con índices modulares. El planificador Round Robin itera desencolando el proceso actual, restándole el *quantum*; si finaliza se retira, de lo contrario vuelve a ingresarse al final de la cola.

C. Resultados y pruebas

La simulación de procesos con distintos tiempos de ejecución y valores *quantum* fue un éxito, verificando la salida:

Plantilla de Informe de Trabajo Práctico

ProcessID	ArrivalTime	BurstTime	TurnaroundTime	WaitingTime
P1	0	5	13	8
P2	2	3	6	3
P3	3	1	4	3
P4	5	2	5	3
P5	6	5	14	9
P6	8	4	11	7

3.3. Problema N° 3: Algoritmos de Ordenamiento

A. Análisis y planteo

- **Descripción abstracta:** Ordenar una secuencia desordenada de N números aleatorios de 5 dígitos en orden ascendente.
- **Operaciones críticas:** Comparación de claves, intercambio de elementos y recorrido de listas.
- **Estructura/Algoritmos elegidos:**
 - **Ordenamiento Burbuja:** Comparaciones e intercambios adyacentes.
 - **Quicksort:** Técnica de "Divide y Vencerás" mediante selección de un pivote.
 - **Radix Sort (LSD):** Procesamiento no comparativo dígito por dígito.
- **Supuestos y justificación:**
 - Burbuja es funcional para entradas muy pequeñas, pero su costo $O(n^2)$ lo vuelve ineficiente al crecer N .
 - Quicksort es óptimo en el caso promedio ($O(n \log n)$).
 - Radix Sort resulta ideal para números con una cantidad fija de dígitos ($d = 5$), ofrece un comportamiento lineal $O(d \cdot n)$.

B. Implementación / Resolución

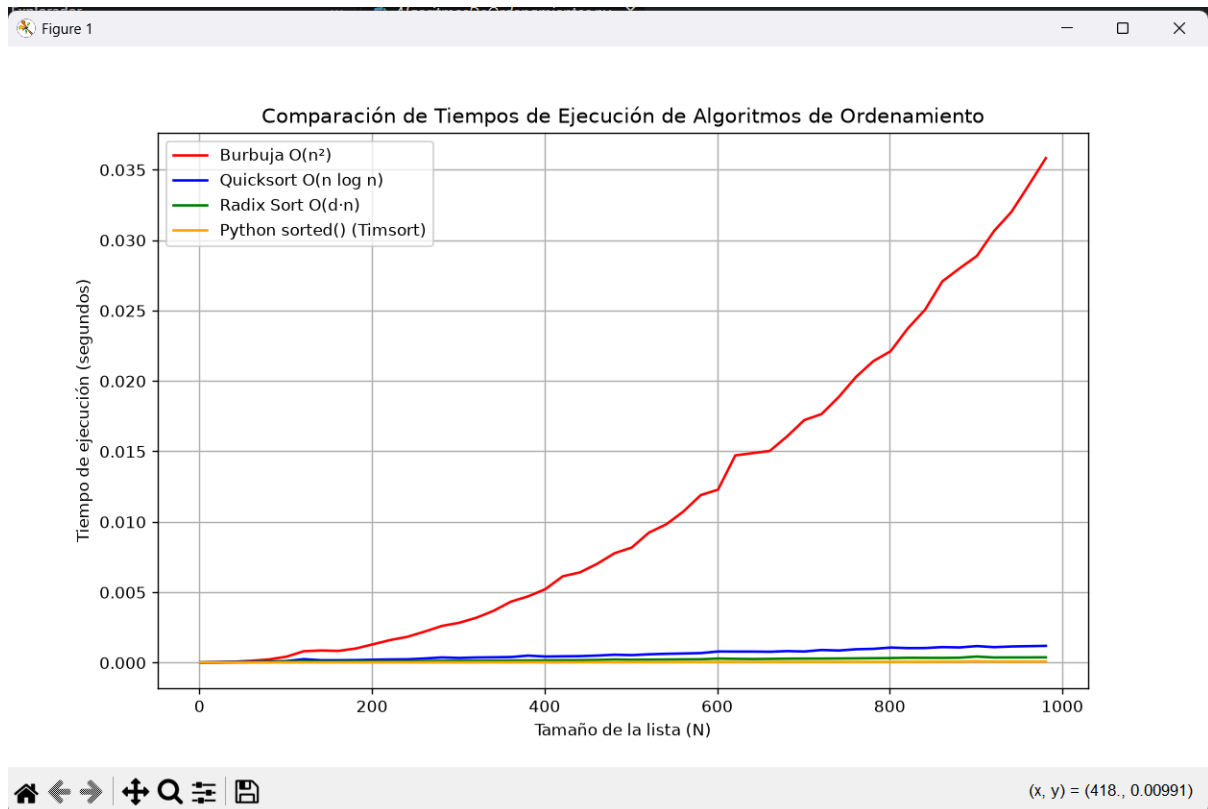
- **Burbuja:** Recorre la lista comparando e intercambiando elementos adyacentes. Incluye una bandera para detenerse si la lista ya está ordenada.
- **Quicksort:** Selecciona un pivote central y divide la lista en tres sublistas (menores, iguales y mayores) aplicando recursión.
- **Radix Sort:** Agrupa los números en 10 urnas (*buckets*) procesando dígito por dígito, desde la unidad hasta la decena de mil.

C. Resultados y pruebas

- **Verificación:** Se probó con 600 números aleatorios de 5 dígitos frente a `sorted()`, obteniendo `True` en los tres algoritmos.
- **Medición de tiempos:** Se midió de $N = 1$ a 1000.
- **Análisis de la gráfica:**
 - **Burbuja ($O(n^2)$):** Curva parabólica con los tiempos más altos.

Plantilla de Informe de Trabajo Práctico

- **Quicksort** ($O(n \log n)$): Crecimiento casi plano, mucho más rápido que Burbuja.
- **Radix Sort** ($O(d \cdot n)$): Crecimiento lineal estable.
- **sorted()** (**Timsort**): El más rápido debido a su implementación híbrida optimizada en lenguaje C.



4. Análisis de resultados específicos / Discusión

Acceso Dinámico vs. Secuencial (Proyecto 1): La Lista Doblemente Enlazada demuestra eficiencia $O(1)$ en inserción/eliminación en extremos, pero costo $O(n)$ en búsquedas, confirmando la ventaja del puntero directo sobre el recorrido secuencial.

Impacto del Quantum (Proyecto 2): Se observa que un *quantum* demasiado pequeño genera alta sobrecarga por cambios de contexto, mientras que un *quantum* muy grande degrada la cola circular a un comportamiento FIFO simple.

Plantilla de Informe de Trabajo Práctico

Comparación Teórica vs. Empírica (Proyecto 3): Los tiempos medidos ratifican la teoría. Burbuja muestra un comportamiento cuadrático $O(n^2)$, mientras que Quicksort y Radix Sort crecen de forma casi plana ($O(n \log n)$ y $O(d \cdot n)$ respectivamente).

Rendimiento de `sorted()` (Timsort): Muestra el mejor desempeño. Al estar optimizado en C y combinar *Insertion Sort* con *Merge Sort*, supera a los algoritmos implementados puramente en Python.

5. Declaración de Uso Ético de Inteligencia Artificial

Durante la elaboración de este trabajo práctico se utilizaron herramientas de Inteligencia Artificial Generativa (ChatGPT / Gemini) con un propósito de apoyo y asistencia académica.

Dado que es nuestro primer acercamiento a este tipo de problemas complejos, sumado a la falta de experiencia previa y a la pérdida de clases prácticas por días feriados y medidas de fuerza docente, se recurrió a estas herramientas para guiar y complementar el proceso de aprendizaje.

- **Alcance del uso:** Se empleó como “tutor virtual” para comprender conceptos teóricos, revisar la notación, asistir en la corrección de errores de código y estandarizar la redacción del informe.
- **Responsabilidad y autoría:** La lógica de la implementación de los algoritmos y estructuras de datos, la ejecución de las pruebas automáticas, la recolección de datos y el análisis crítico de las gráficas fueron realizados y validados de manera personal por los autores del trabajo.

6. Trabajo en equipo

7. Conclusiones

Cumplimiento de objetivos: Se lograron implementar, validar y analizar con éxito las tres soluciones propuestas (Lista Doblemente Enlazada, Cola Circular para Round Robin y los tres algoritmos de ordenamiento). Se verificó que las mediciones empíricas coinciden con la complejidad teórica calculada *a priori*.

Dificultades y resolución: Las principales dificultades fueron el manejo de punteros en la LDE para no perder referencias, el control de índices modulares en la cola circular y la

Plantilla de Informe de Trabajo Práctico

recolección precisa de tiempos de ejecución para listas grandes. Se superaron mediante pruebas unitarias paso a paso y el uso de librerías de medición precisas (`time.perf_counter`).

Aprendizajes: Se comprendió la importancia de elegir la estructura de datos adecuada según el problema. Además, se comprobó experimentalmente por qué los algoritmos de ordenamiento eficientes ($O(n \log n)$ y $O(d \cdot n)$) son fundamentales para procesar grandes volúmenes de información.

8. Referencias y Bibliografía

Miller, B., & Ranum, D. (2013). Solución de problemas con algoritmos y estructuras de datos usando Python (M. Orozco-Alzate, Trad.; 2.a ed.).

Material Teórico de la Cátedra: Diapositivas y apuntes de las unidades temáticas de Algoritmos y Estructuras de Datos..

INFORME INCOMPLETO, FALTA COMPLETAR PUNTO 6